ROLL NO : 241801038

Ex14: Write a code using RAW sockets to implement packet sniffing.

Aim :

To write a program using RAW sockets to capture and display network packets passing through the network interface.

Algorithm :

1. Start the program.
2. Create a RAW socket using socket(AF_PACKET, SOCK_RAW, ...).
3. Bind the socket to the required network interface.
4. Continuously receive packets from the network.
5. Extract the Ethernet header from each packet.
6. Identify the protocol type, such as IPv4, ARP, or IPv6.
7. For IPv4 packets, extract the source and destination IP addresses.
8. Display the packet information, including MAC addresses, protocol, IP addresses, TTL, and IP protocol.
9. Repeat the process until the user terminates the program.
10. Close the RAW socket.
11. Stop the program.

Code :

```
import socket
import struct

# Create a RAW socket
raw_socket = socket.socket(
    socket.AF_PACKET,
    socket.SOCK_RAW,
    socket.ntohs(3)
```

```
    )

    print("Packet Sniffer Started...")
    print("Waiting for packets...\n")

    try:
        while True:
            # Receive packet
            packet, address = raw_socket.recvfrom(65535)

            # Ethernet header is first 14 bytes
            ethernet_header = packet[:14]

            # Unpack Ethernet header
            dest_mac, src_mac, protocol = struct.unpack(
                "!6s6sH", ethernet_header
            )

            # Convert MAC addresses
            dest_mac = ":".join(
                "%02x" % byte for byte in dest_mac
            )
            src_mac = ":".join(
                "%02x" % byte for byte in src_mac
            )

            print("==================================")
            print("Source MAC      :", src_mac)
            print("Destination MAC :", dest_mac)
            print("Protocol        :", hex(protocol))

            # IPv4 packet
            if protocol == 0x0800:
                ip_header = packet[14:34]

                version_ihl, tos, total_length, identification, \
                flags_fragment, ttl, proto, checksum, \
                src_ip, dest_ip = struct.unpack(
                    "!BBHHHBBH4s4s",
                    ip_header
                )

                src_ip = socket.inet_ntoa(src_ip)
                dest_ip = socket.inet_ntoa(dest_ip)

                print("Source IP       :", src_ip)
                print("Destination IP  :", dest_ip)
                print("TTL             :", ttl)
                print("IP Protocol     :", proto)

            elif protocol == 0x0806:
```

```
                print("Packet Type     : ARP")

            elif protocol == 0x86DD:
                print("Packet Type     : IPv6")

            else:
                print("Packet Type     : Other")

    except KeyboardInterrupt:
        print("\nPacket sniffing stopped.")

    finally:
        raw_socket.close()
```

```
Streaming output truncated to the last 5000 lines.
Destination MAC : 00:00:00:00:00:00
Protocol        : 0x800
Source IP       : 172.28.0.12
Destination IP  : 172.28.0.12
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 00:00:00:00:00:00
Destination MAC : 00:00:00:00:00:00
Protocol        : 0x800
Source IP       : 172.28.0.12
Destination IP  : 172.28.0.12
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 02:42:ac:1c:00:0c
Destination MAC : 02:42:ac:de:c7:45
Protocol        : 0x800
Source IP       : 172.28.0.12
Destination IP  : 172.28.0.1
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 02:42:ac:de:c7:45
Destination MAC : 02:42:ac:1c:00:0c
Protocol        : 0x800
Source IP       : 172.28.0.1
Destination IP  : 172.28.0.12
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 00:00:00:00:00:00
Destination MAC : 00:00:00:00:00:00
Protocol        : 0x800
Source IP       : 172.28.0.12
Destination IP  : 172.28.0.12
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 00:00:00:00:00:00
```

```
Destination MAC : 00:00:00:00:00:00
Protocol        : 0x800
Source IP       : 172.28.0.12
Destination IP  : 172.28.0.12
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 00:00:00:00:00:00
Destination MAC : 00:00:00:00:00:00
Protocol        : 0x800
Source IP       : 172.28.0.12
Destination IP  : 172.28.0.12
TTL             : 64
IP Protocol     : 6
==================================
Source MAC      : 00:00:00:00:00:00
Destination MAC : 00:00:00:00:00:00
```

Result :

Thus, a packet-sniffing program using RAW sockets was successfully implemented. The program captures network packets and displays important information such as source MAC address, destination MAC address, protocol, source IP address, destination IP address, and TTL.